kafka-isotope

End-to-end **record tracing for Apache Kafka**, built entirely on public extension points — a `ProducerInterceptor` plus record headers. No broker changes, no agent, no vendor lock-in. Stamp a UUIDv7 trace id and origin metadata on first produce, accumulate a hop on every re-produce, and reconstruct latency / topology / hop-distribution from the headers (or from the optional Prometheus meters).

Modules

Module	Coordinate	Pulls	Use it for
isotope-core	`ai.signalroom:isotope-core`	Jackson, SLF4J (`kafka-clients` is `compileOnly`)	Trace **propagation** — interceptor, headers, consume markers.
isotope-metrics	`ai.signalroom:isotope-metrics`	isotope-core + Micrometer/Prometheus	Optional `/metrics` exporter for the stateless reports.

`isotope-core` never depends on a metrics library: emission routes through a no-op `IsotopeMetricsSink` until `isotope-metrics` registers the Prometheus one, so propagation runs with zero metrics overhead.

Install (Gradle, GitHub Packages)

```
repositories {
    maven { url 'https://maven.pkg.github.com/j3-signalroom/kafka-isotope'
}
}
dependencies {
    implementation 'ai.signalroom:isotope-core:0.17.1'
    implementation 'ai.signalroom:isotope-metrics:0.17.1' // optional —
only for Prometheus
}
```

See **isotope-core/README.md** for the full adopter quickstart (registering the interceptor, adopting/marking on consume, starting the exporter).

Build

```
./gradlew build                  # compile + test both modules
./gradlew publishToMavenLocal    # install 0.17.1 into ~/.m2 for local
consumers
```

The library targets **Java 17** bytecode for wide adoption (toolchain builds on 21).

Publish to GitHub Packages

```
GITHUB_ACTOR=<user> GITHUB_TOKEN=<token-with-write:packages> \
  ./gradlew publishAllPublicationsToGitHubPackagesRepository
```

Credentials may instead come from the `gpr.user` / `gpr.key` Gradle properties. Note: GitHub Packages requires consumers to authenticate even for public artifacts — Maven Central (below) does not.

Publish to Maven Central (Sonatype Central Portal)

Frictionless public consumption — no auth to depend on it. One-time prerequisites:

1. **Verify the `ai.signalroom` namespace** at central.sonatype.com (a DNS TXT record on `signalroom.ai`), and generate a **publisher user token**.
2. **Create a GPG key**, publish it to a keyserver (`gpg --full-generate-key`; `gpg --keyserver hkps://keyserver.ubuntu.com --send-keys <id>` — use `hkps://` to avoid the often-blocked HKP port 11371), and export the private key:

   ```
   gpg --batch --pinentry-mode loopback --passphrase '<gpg-passphrase>' \
       --export-secret-keys --armor <id>
   ```

   On GnuPG 2.4+ (`keyboxd`), a plain `gpg --export-secret-keys --armor <id>` run non-interactively (piped, in a subshell, or in CI) emits **nothing** when the agent can't prompt for the passphrase — and a downstream signing step then fails with `Could not read PGP secret key` (`tag 0xffffffff`). The `--pinentry-mode loopback --passphrase …` form forces the unlock and produces the real ~7 KB armored block.

Then publish (signing turns on automatically once the key is present):

```
export ORG_GRADLE_PROJECT_mavenCentralUsername=<central-token-user>
export ORG_GRADLE_PROJECT_mavenCentralPassword=<central-token-pass>
export ORG_GRADLE_PROJECT_signingInMemoryKey="$(gpg --batch --pinentry-mode
loopback \
  --passphrase '<gpg-passphrase>' --export-secret-keys --armor <id>)"
export ORG_GRADLE_PROJECT_signingInMemoryKeyPassword=<gpg-passphrase>

./gradlew publishAndReleaseToMavenCentral
```

`SONATYPE_HOST=CENTRAL_PORTAL` and `SONATYPE_AUTOMATIC_RELEASE=true` (in `gradle.properties`) target the Central Portal and auto-release once validation passes. Without a signing key, signing is skipped — so `build` / `publishToMavenLocal` / GitHub Packages stay key-free; only the Central path requires it.

Automated release via CI

`.github/workflows/publish.yml` runs the same `publishAndReleaseToMavenCentral` task on GitHub Actions. Add these four repository secrets (**Settings → Secrets and variables → Actions**):

Secret	Value
`MAVEN_CENTRAL_USERNAME`	Central Portal publisher token username
`MAVEN_CENTRAL_PASSWORD`	Central Portal publisher token password
`SIGNING_KEY`	base64 of the armored key: `gpg --batch --pinentry-mode loopback --passphrase '<pass>' --export-secret-keys --armor <id> | base64`
`SIGNING_KEY_PASSWORD`	the GPG passphrase that unlocks `SIGNING_KEY`

> **Store `SIGNING_KEY` base64-encoded, not as a raw paste.** GitHub's secret store mangles the armored key's line breaks, after which Gradle fails with `Could not read PGP secret key`. Base64 keeps it a single line; the workflow decodes it back to the armored key before signing.

The published version comes from the **release tag** (a leading `v` is stripped, so `v0.17.1` → `0.17.1`), overriding `version` in `gradle.properties` — the tag is the single source of truth. Publish a GitHub Release tagged `v0.17.1` to ship it, or trigger the workflow manually from the Actions tab and supply the version.

Demo

A runnable reference pipeline (Confluent Platform / Confluent Cloud on Minikube, the seven Flink reports, and a one-command Prometheus + Grafana showcase) lives in the companion repo: j3-signalroom/confluent-kafka-isotope.